



INSTITUTO TECNOLÓGICO DE CD. VICTORIA

Carrera: Ingeniería en Sistemas Computacionales

Materia: Graficación

Unidad 2 Tarea 1.- Transformación en 2D

Núm. De Equipo: #3

Integrantes del Equipo:

Sebastián Emiliano García Reyes 23380387

Javier Alexis Rodríguez Báez 23380417

Pedro Vázquez Maldonado 21380442

Cristian Yahir Hernández De la Cruz 21380393

Eder Samuel Alvarado Molina 20380008

Itzel Alejandra Paz Chagolla 23380412

Yahir Misael Lores De Anda 22380401

Luis Gerardo Macias Rodríguez 21380400

Karol Zayde Carrizales Álvarez 23380375

Profesor: Jorge Luis Reyes Cedillo.

Grupo: 72

Cd. Victoria, Tam., 12 de octubre de 2025



Introducción

La aplicación realizada es una herramienta de software, cuyo propósito principal es la visualización y experimentación con transformaciones geométricas afines en un entorno bidimensional (2D). Ha sido diseñada para servir como una plataforma de aprendizaje, permitiendo a los usuarios aplicar operaciones de traslación, escalado, rotación y sesgado a un objeto gráfico y observar los resultados en tiempo real. El alcance del sistema se centra en la demostración de estos conceptos fundamentales de la computación gráfica, más que en proveer funcionalidades de una herramienta de diseño gráfico de producción. Su valor reside en la claridad con la que expone la relación causa-efecto entre los parámetros de una transformación y su impacto visual en la geometría del objeto.

La aplicación está desarrollada en el lenguaje de programación Java, una elección motivada por su robusto paradigma de programación orientada a objetos y la portabilidad que ofrece la Máquina Virtual de Java. La POO es fundamental para la arquitectura del sistema, ya que permite modelar los componentes del programa como objetos que encapsulan estado y comportamiento, facilitando la modularidad y el mantenimiento del código.

Para la construcción de la Interfaz Gráfica de Usuario, el sistema emplea la biblioteca **Java Swing**. Swing es un framework que proporciona un conjunto de componentes para crear aplicaciones de escritorio interactivas.

El núcleo del renderizado gráfico se apoya en la **API Java 2D**, un conjunto de clases e interfaces para el dibujo avanzado de gráficos bidimensionales. Esta API, que incluye clases fundamentales como Graphics2D, AffineTransform y Path2D, ofrece un control de bajo nivel sobre el proceso de renderizado, permitiendo la creación de geometrías complejas, la aplicación de transformaciones sofisticadas y el control de atributos de renderizado como el antialiasing para mejorar la calidad visual.



Arquitectura de Componentes

La arquitectura se fundamenta en una clara separación de responsabilidades, materializada en dos clases principales: `Transformaciones_2D` y `DrawingPanel`. Este diseño sigue los principios de la programación orientada a objetos para lograr un sistema modular y cohesivo.

- **Transformaciones_2D:** Esta clase actúa como el orquestador principal de la aplicación y el controlador de la interfaz de usuario. Hereda de `JFrame` para funcionar como la ventana principal.
- **DrawingPanel:** Esta clase, que hereda de `JPanel`, es el componente central de la aplicación. Encapsula toda la lógica relacionada con el estado del objeto gráfico y su renderizado.

Esta separación constituye una decisión clave que va más allá de la simple organización del código. La comunicación entre estos dos componentes es estrictamente unidireccional: Tiene una referencia a `DrawingPanel` y puede invocar sus métodos públicos, pero `DrawingPanel` no tiene conocimiento alguno sobre la existencia o la estructura de `Transformaciones_2D`.

Este diseño de bajo acoplamiento convierte a `DrawingPanel` en un componente de software autónomo y altamente reutilizable. Podría ser extraído de esta aplicación e integrado en cualquier otro contenedor Swing con un esfuerzo mínimo, ya que su funcionalidad no depende de la interfaz de usuario específica que lo controla.



Modelo de Interacción

La interactividad de la aplicación se rige por el modelo de eventos estándar de Java Swing. El flujo de control se inicia con una acción del usuario y sigue una secuencia bien definida:

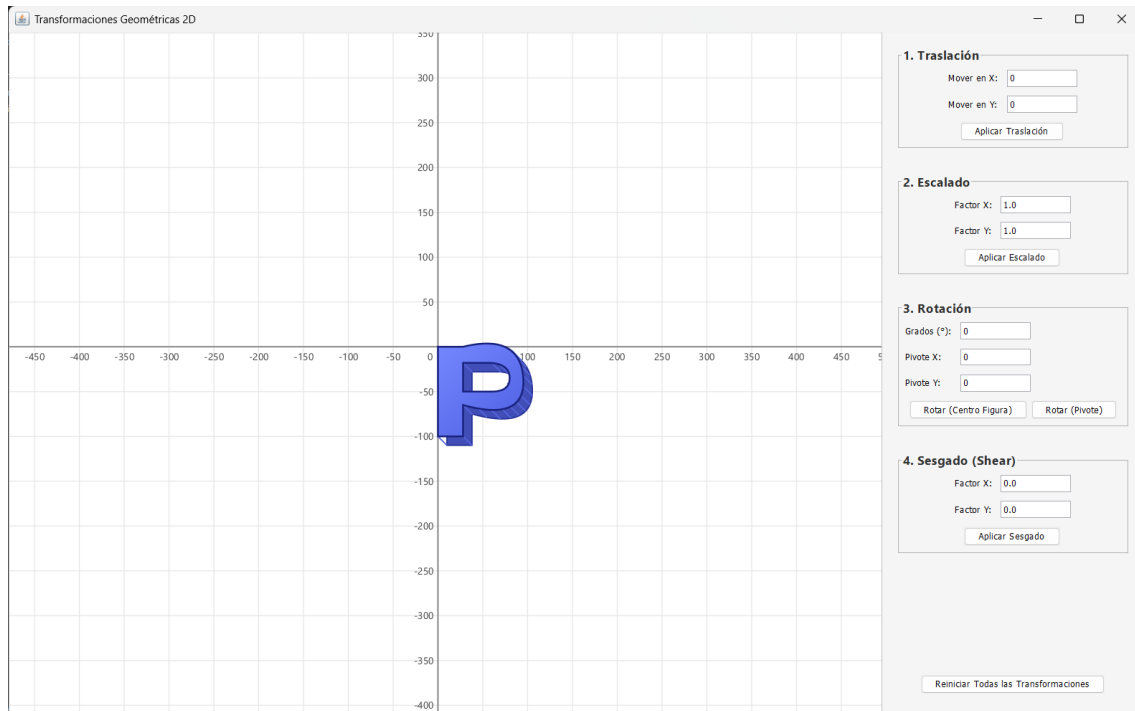
1. **Generación del Evento:** El usuario interactúa con un componente de la GUI, como hacer clic en un botón. Esto hace que el componente genere un objeto de evento.
2. **Captura del Evento:** Se han registrado ActionListeners en cada botón.
3. **Procesamiento del Evento:** El código dentro del ActionListener se ejecuta. Típicamente, este lee los valores de los JTextField correspondientes, los convierte a tipos numéricos y los utiliza para invocar un método público en la instancia de drawingPanel.
4. **Actualización del Estado:** La llamada al método en DrawingPanel modifica su estado interno. Específicamente, actualiza su matriz AffineTransform para incorporar la nueva transformación.
5. **Solicitud de Repintado:** Inmediatamente después de actualizar su estado, el método de transformación invoca a repaint(). Esta llamada no dibuja directamente, sino que notifica al sistema de gestión de eventos de Swing que el componente DrawingPanel necesita ser redibujado lo antes posible.
6. **Ciclo de Renderizado:** El sistema Swing, cuando lo considera oportuno, invoca al método paintComponent(Graphics g) de DrawingPanel. Este método es el responsable final de dibujar el estado actual del objeto, aplicando la matriz de transformación acumulada a la geometría original y renderizándola en el panel.

Este ciclo garantiza que la interfaz de usuario permanezca receptiva mientras que la lógica de renderizado se ejecuta de manera eficiente solo cuando es necesario.

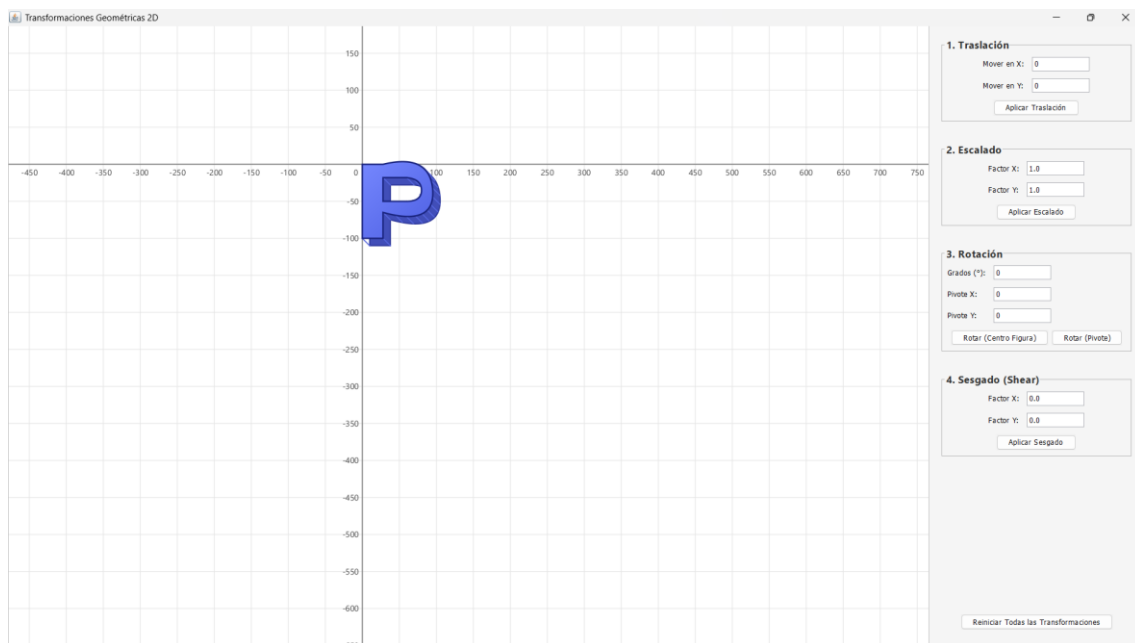
Análisis Específico de Cada Transformación

- **Traslación:** La implementación en `translate(tx, -ty)` es directa, creando una matriz de traslación. Nótese el `-ty`, que es necesario para mapear el sistema de coordenadas de la UI (donde Y aumenta hacia abajo) al sistema de coordenadas cartesiano estándar (donde Y aumenta hacia arriba).
- **Escalado:** El método `scale(sx, sy)` implementa el escalado con respecto al centro del objeto, no al origen. Esto se logra mediante una composición de tres transformaciones:
 1. Trasladar el objeto para que su centro coincida con el origen.
 2. Aplicar el escalado deseado.
 3. Trasladar el objeto de vuelta a su posición original.El código encapsula estas tres operaciones en una única `AffineTransform (scaleTransform)` que luego se pre-concatena a la transformación global.¹⁶
- **Rotación:** Se proporcionan dos métodos de rotación. `rotate(degrees)` implementa la rotación alrededor del centro del objeto, siguiendo la misma lógica de "trasladar-rotar-trasladar" que el escalado. `rotate(degrees, pivotX, pivotY)` permite la rotación alrededor de un punto de pivote arbitrario especificado por el usuario, lo que demuestra la flexibilidad de la API `AffineTransform`.
- **Sesgado (Shear):** El método `shear(shx, shy)` es la aplicación más directa, ya que el sesgado se define típicamente con respecto al origen, y el efecto visual es el mismo independientemente de la posición del objeto.

Vista del programa

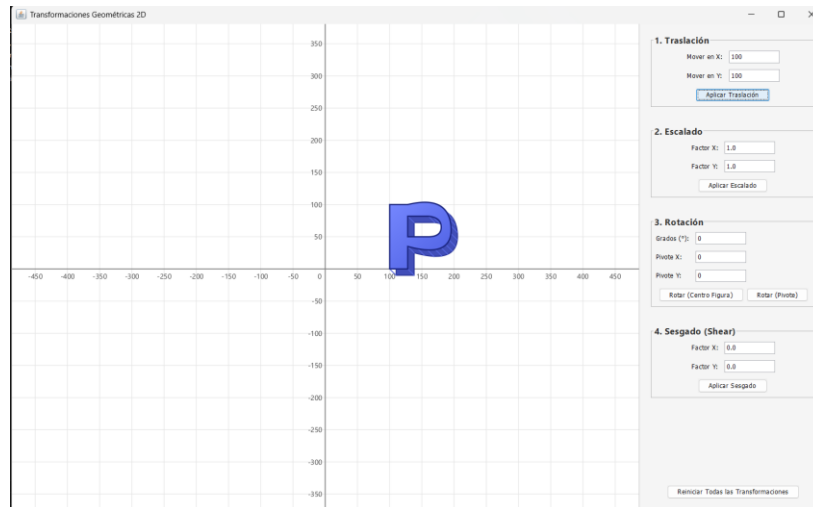


El programa se encuentra desarrollado para que el usuario pueda mover el plano para mejor visualización en caso de hacer mas grande la letra o se mueva de lugar con traslación



Tambien se agrego que se pueda maximizar la pantalla sin que la visualización colapse.

Traslación



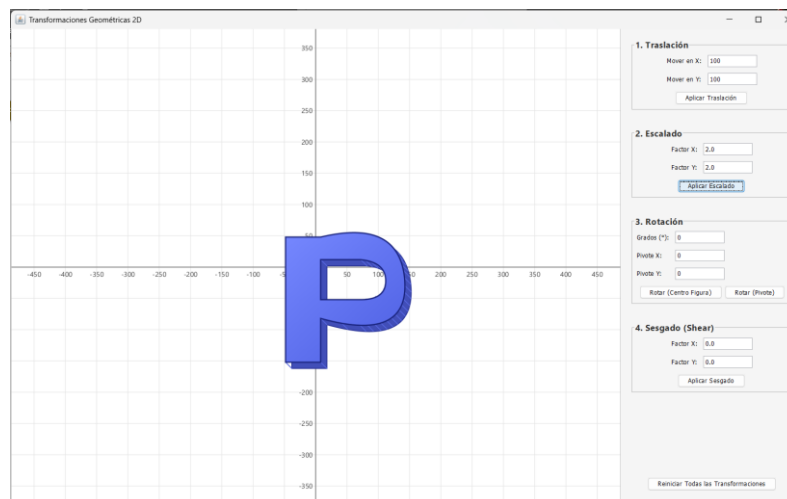
La traslación desplaza un objeto a una nueva posición en el plano. Se define por un vector de traslación.

En la implementación, se utiliza `translate(tx, -ty)`. El uso de `-ty` es crucial para mapear el sistema de coordenadas de la UI de Java (donde el eje Y incrementa hacia abajo) al sistema de coordenadas cartesiano estándar (donde Y incrementa hacia arriba).

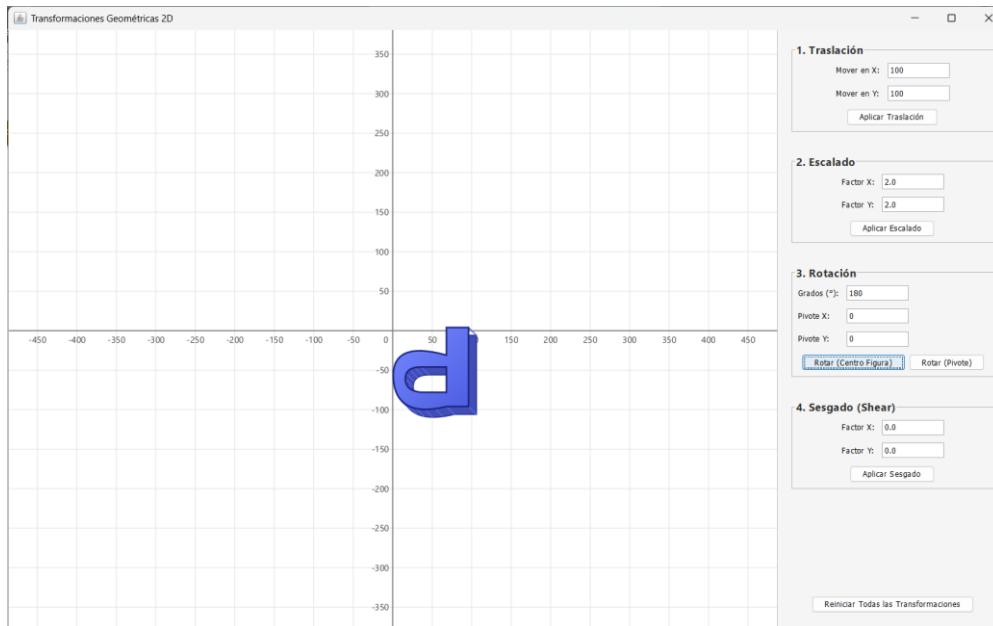
Escalado

El escalado altera el tamaño del objeto. Se implementa con respecto al centro del objeto para un efecto más intuitivo. Esto se logra mediante una composición de tres transformaciones:

1. Trasladar el objeto para que su centro coincida con el origen de coordenadas.
2. Trasladar el objeto de vuelta a su posición original.

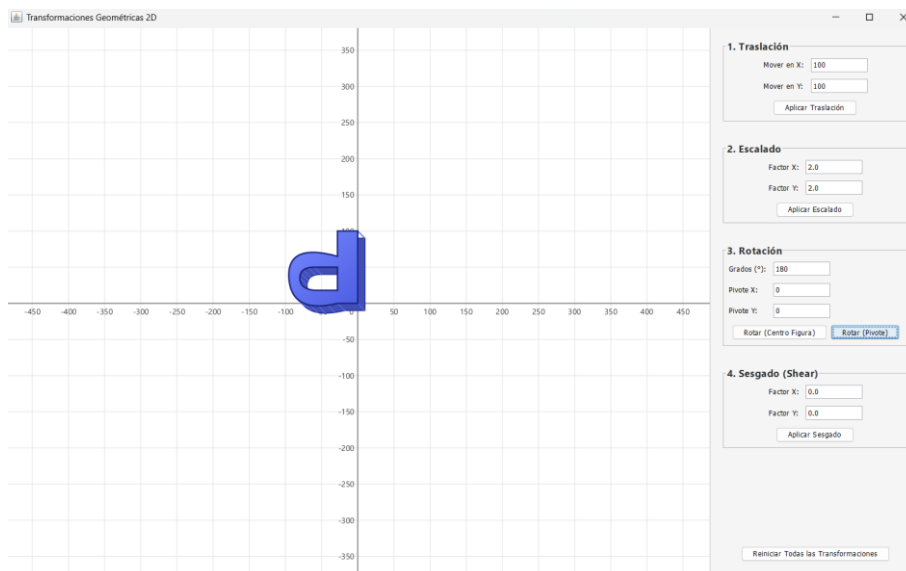


Rotación



La rotación gira el objeto un ángulo θ . La aplicación ofrece dos modalidades:

- Rotación sobre el centro del objeto: Sigue la misma lógica de "trasladar-rotar-trasladar" que el escalado.

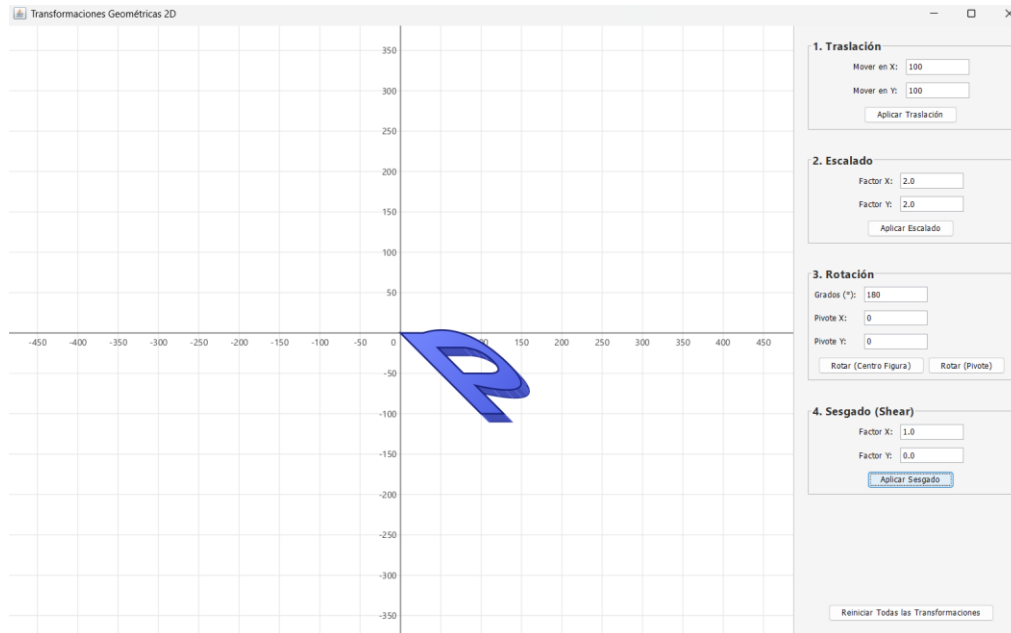


- Rotación sobre un pivote arbitrario: Permite al usuario especificar un punto como centro de rotación, demostrando la flexibilidad de la API.

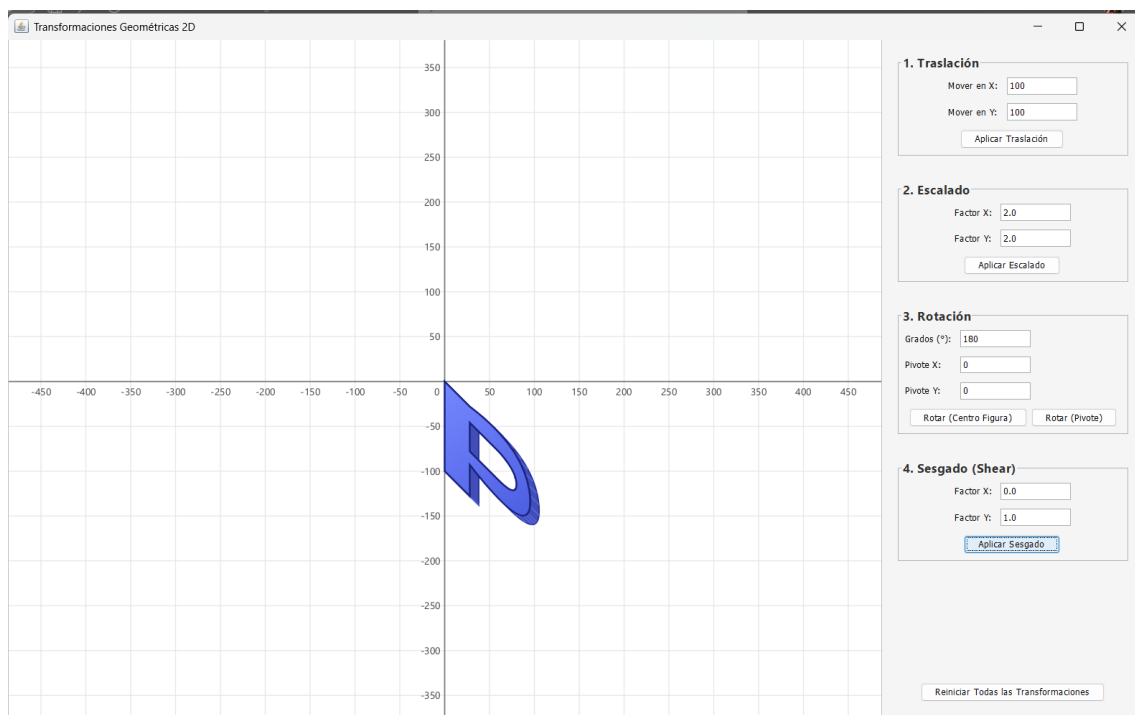
Sesgaso

El sesgado deforma el objeto, inclinándolo a lo largo de los ejes X o Y.

Sesgaso (X)



Sesgado(X): Un factor de 1.0 en X inclina la figura horizontalmente.



Sesgado (Y): De manera similar, un factor de 1.0 en Y produce una deformación vertical.



Bibliografías

- Computer Graphics - 2D Transformation. (s.f.). *Tutorials Point*. Acceso: octubre 12, 2025, de https://www.tutorialspoint.com/computer_graphics/2d_transformation.htm
- *Desarrollo de software orientado a objetos*. (s.f.). Ginzo Technologies. Acceso: octubre 12, 2025, de <https://ginzo.tech/desarrollo-de-software-orientado-a-objetos/>
- Gálvez, J. (s.f.). *Java a tope: Java2D*. Departamento de Lenguajes y Ciencias de la Computación. Acceso: octubre 12, 2025, de <http://www.lcc.uma.es/~galvez/ftp/libros/Java2D.pdf>
- GraphicMaths. (s.f.). *2D transformation matrices*. Acceso: octubre 12, 2025, de <https://graphicmaths.com/pure/matrices/matrix-2d-transformations/>
- IBM. (s.f.). *Programación orientada a objetos*. Acceso: octubre 12, 2025, de <https://www.ibm.com/docs/es/spss-modeler/saas?topic=language-object-oriented-programming>
- Intelequia. (s.f.). *Qué es la Programación Orientada a Objetos*. Acceso: octubre 12, 2025, de <https://intelequia.com/es/blog/post/qu%C3%A9-es-la-programaci%C3%B3n-orientada-a-objetos>
- MOOC.fi. (s.f.). *Class diagrams*. Java Programming. Acceso: octubre 12, 2025, de <https://java-programming.mooc.fi/part-11/1-class-diagrams/>
- NAppGUI. (s.f.). *Transformaciones 2D*. SDK Multiplataforma en C. Acceso: octubre 12, 2025, de <https://nappgui.com/es/geom2d/t2d.html>
- Oracle. (s.f.). *AffineTransform (Java Platform SE 8)*. Oracle Help Center. Acceso: octubre 12, 2025, de <https://docs.oracle.com/javase/8/docs/api/java/awt/geom/AffineTransform.html>
- Oracle. (s.f.). *PathIterator (Java Platform SE 8)*. Oracle Help Center. Acceso: octubre 12, 2025, de <https://docs.oracle.com/javase/8/docs/api/java/awt/geom/PathIterator.html>
- Peinado, F. (s.f.). *Laboratorio de Programación de Sistemas – Ing. Software Orientada a Objetos*. Universidad Complutense de Madrid. Acceso: octubre 12, 2025, de <https://www.fdi.ucm.es/profesor/fpeinado/courses/oop/LPS-03IngSoftwareOrientadaaObjetos.pdf>
- Shene, C. (s.f.). *Geometric Transformations*. Michigan Technological University. Acceso: octubre 12, 2025, de <https://pages.mtu.edu/~shene/COURSES/cs3621/NOTES/geometry/geo-tran.html>